ANTIGRAVITY HISTORY

Forensic Postmortem & Recovery Engineering Report

A full-depth reconstruction of the investigation into missing Conversation History/Search entries, the evidence that separated the durable conversation store from the History projection, and the engineering controls used to restore and preserve consistency.

Item	Value
Incident	Antigravity History/Search divergence
Date	2026-10-03
Authoritative evidence	58 readable SQLite conversation databases
Broken projection	antigravityUnifiedStateSync.trajectorySummaries
Affected example	c99ca0b3-a658-48ae-9dba-33eed0571a99
Recovery	58 DB-derived summaries reconstructed; 24 orphan IDs removed
Permanent control	External idempotent reconciliation at logon + hourly

Document purpose

This is an engineering forensic record, not a short incident summary. It preserves the investigation's tool changes, false leads, source/binary tracing, exact data relationships, recovery decisions, validation, and remaining caveats. Statements are classified as observed/proven, disproven, or unsupported where appropriate.

1 — Executive Summary

The visible symptom was selective disappearance from Antigravity Conversation History/Search. The underlying conversation files had not disappeared. The decisive finding was that two different persistence layers had diverged: the durable conversation databases remained healthy, while the History summary projection had become incomplete and stale.

The active IDE conversation directory contained 58 SQLite databases. Every database passed SQLite integrity checking. The affected conversation c99ca0b3-a658-48ae-9dba-33eed0571a99 was readable and contained 329 steps. In contrast, the persisted trajectorySummaries map contained 25 entries. Only one of those 25 corresponded to a current DB; 24 were orphan identifiers, while 57 valid DBs had no entry in the persisted map.

The investigation moved through several environments. It began with direct PowerShell/filesystem inspection, then used Antigravity CLI to understand the local application structure. When the problem became a source/binary archaeology task, the investigation shifted to OpenCode/Big Pickle. Kiro and Codex/Work were also used as supplementary paths but were constrained by usage/token limits. The final conclusions came from installed bundles, native binary behavior, database evidence, controlled reconstruction, and restart/fault-injection tests.

A major lesson was that early agent explanations could not be treated as root-cause evidence. Several confident explanations were later disproven: the 25 entries were not a 25-entry eviction policy; antigravity-cli was not established as the IDE's source; migration did not explain steady-state repair; and CascadeManager was not bulk-hydrated at startup.

The final root cause is an event-derived, write-only History summary snapshot with no steady-state reconciliation against the authoritative conversation store. The summary map is seeded from its previous persisted value, mutated by runtime events, and written as a whole map at shutdown. No normal startup path was found that enumerates all conversation DBs and rebuilds missing summaries.

The permanent solution was deliberately external to the vendor code: an idempotent reconciler enumerates the authoritative DBs, derives the summary representation, removes ghosts, rebuilds damaged entries, backs up before writes, refuses to run while Antigravity is active, and runs at logon and hourly.

2 — Incident Scope and Initial Evidence

2.1 What was actually missing?

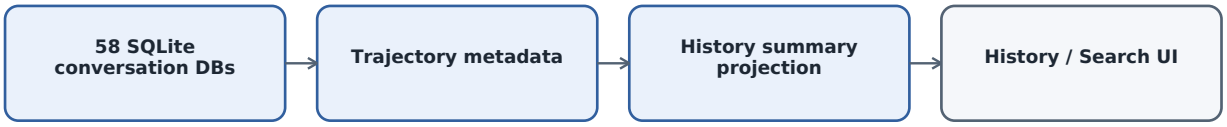
The first forensic question was not 'why is History broken?' but 'is the conversation itself gone?' That distinction prevented premature repair of the wrong layer.

Evidence	Observation	Meaning
Conversation DBs	58 present; quick_check OK	Durable conversation data survived
Legacy PB files	73 present	Older storage generation
Persisted summary map	25 entries	History projection incomplete
Current DB IDs in summary	1 / 58	Almost no current-store coverage
Orphan summary IDs	24	Projection retained stale/nonexistent records
Affected c99 DB	329 steps	Healthy, readable source

2.2 Evidence hierarchy

The recovery boundary was set at the deepest durable source available. The History UI was treated as a projection, not as the source of truth. This became the foundation for every later repair decision.

Evidence hierarchy



3 — The Investigation Journey

The investigation was deliberately iterative. Each tool was used for a different level of evidence, and the working hypothesis changed when artifacts contradicted it.

Approx. phase	Activity	Result
~04:30	PowerShell / local inspection	Found conversation DBs despite missing UI entries
~05:00	Antigravity CLI	Located relevant application roots, bundles and symbols
~06:00	Shift to OpenCode / Big Pickle	Enabled deeper repository-style source/binary tracing
~06:30	Installed bundle + binary archaeology	Mapped History RPC and runtime loading paths
~07:30	Summary lifecycle tracing	Found seed → event update → shutdown persistence
~09:00	Recovery construction	Rebuilt projection from 58 DBs
~10:30	Fault injection + restart	Proved convergence and durability
~11:00	User UI test	Confirmed repaired History behavior

3.1 Phase I — Manual filesystem investigation

Direct Windows inspection established the most important fact early: conversations that the UI could not find still existed as individual .db files. This immediately made simple deletion an insufficient explanation and redirected the investigation toward discovery, indexing, loading, or projection persistence.

3.2 Phase II — Antigravity CLI

Antigravity CLI was valuable for initial navigation and architecture discovery. It exposed relevant names such as trajectorySummaries, SearchConversations, GetAllCascadeTrajectories, Trajectory, trajectory_meta, trajectory_metadata_blob, and trajectory-store components.

However, its early explanations were treated as hypotheses. Several were later contradicted by executable/source evidence. The investigation therefore changed from 'ask the agent what is happening' to 'trace the exact state transition and verify it against the artifact.'

3.3 Phase III — Shift to OpenCode / Big Pickle

OpenCode became the main forensic workbench once the task required broad searches across installed bundles, call-site tracing, symbol analysis, and repeated testing of competing hypotheses. Kiro and Codex/Work contributed supplementary reasoning but were interrupted by token/usage limits. The final conclusions were not dependent on their unavailable continuations.

3.4 Why this shift mattered

The critical change was methodological: plausible explanations stopped being accepted without a chain of evidence. The final standard became file → function → caller → state transition → persistence boundary → observed behavior.

4 — Hypothesis Register

The early investigation produced several plausible models. Keeping the rejected models in the record is important because they explain why the investigation moved through multiple layers.

Hypothesis	Status	Evidence
25-entry cache / eviction	DISPROVEN	Cleanup cap is 100; entries were not simply the newest 25.
state.vscdb is the conversation store	DISPROVEN / REFINED	It stores the History summary projection; DBs are the durable conversation source.
antigravity-cli root is corrupting IDE History	NOT SUPPORTED	IDE and CLI use distinct mode/app-data configuration.
Migration guard prevented normal History open	DISPROVEN	Migration is one-shot; no steady-state DB reconciliation was established.
Startup bulk-loads every trajectory	DISPROVEN	CascadeManager is load-driven; initialization registers store information.
CascadeManager owns the durable History index	DISPROVEN	Its GetAllTrajectories path reads an in-memory registry.
Workspace filtering explains the data loss	INSUFFICIENT	Filtering explains visibility, not 57 DBs missing from persisted summary state.
Summary projection lacks DB reconciliation	PROVEN	No steady-state authoritative DB scan/rebuild path was found.

The 25-entry clue

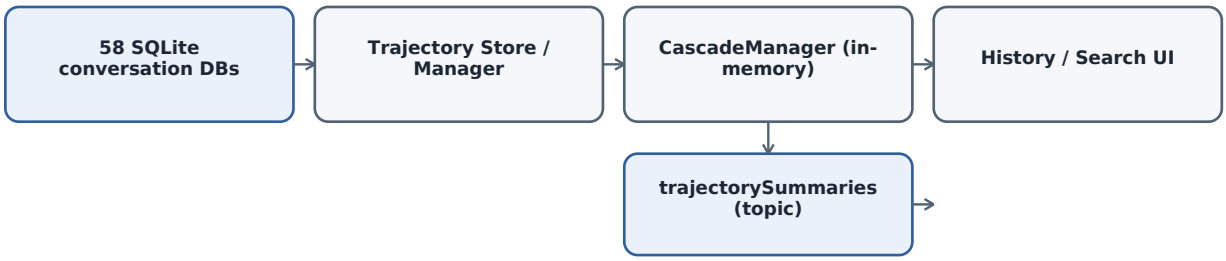
The number 25 looked suspicious because many systems use bounded history caches. That was a reasonable investigative clue. It was not a conclusion. Direct inspection showed the cleanup threshold is 100 and that cleanup sorts by the summary's own lastModifiedTime. With only 25 entries, the cleanup mechanism cannot explain why 57 valid DBs were absent.

This was the first major example of the investigation correcting an attractive but incorrect story: the number of entries was evidence about the state, not proof of the policy that produced it.

5 — Architecture Reconstruction

The final architecture has three distinct concerns: the durable conversation store, the runtime CascadeManager registry, and the separately persisted History summary projection.

Recovered architecture



5.1 History RPC path

This path does not enumerate the conversation directory. GetAllTrajectories operates on the CascadeManager's current in-memory registry. That registry is not equivalent to the complete set of DB files on disk.

5.2 Runtime loading path

This explains why a valid DB can remain invisible to a fresh process: it does not imply that its trajectory has been loaded into the in-memory registry. Runtime loading is demand-driven.

6 — The Decisive Breakthrough: Summary Lifecycle

The root cause became clear only after tracing how trajectorySummaries is initialized, mutated, and persisted.

History summary lifecycle



6.1 Seed

Mss.initialize starts from the existing persisted topic state, with a legacy copy path gated by a migration flag. This is a snapshot seed. It is not a database reconciliation pass.

6.2 Mutation

The central mutator is pushUpdate. It performs incremental map changes. Create can introduce a new cascadeId; delete removes one. Other update types require an existing state entry.

6.3 Persistence

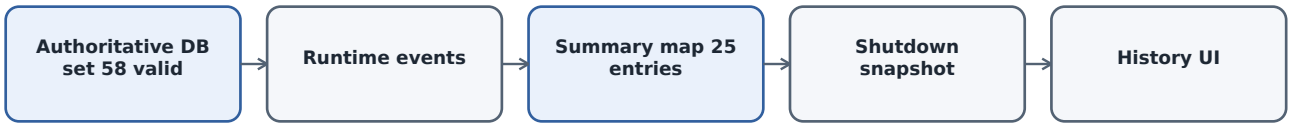
The entire topic is serialized during shutdown. The operation stores the whole map; it does not merge it against the conversation directory.

Therefore, an incomplete runtime map can become a durable incomplete snapshot, and an orphan entry can remain durable because there is no foreign-key-like validation against the DB set.

7 — Root Cause

ROOT CAUSE: The History/Search projection is maintained as an event-derived, write-only summary snapshot and is not continuously reconciled against the authoritative conversation database set.

This mechanism explains both sides of the observed failure. Missing real entries persist because nothing necessarily emits a create event for a conversation that exists on disk but was never loaded/registered. Ghost entries persist because the summary layer does not validate every cascadeId against the current DB set before persistence.



Observed fact	Root-cause mechanism
57 real DBs absent	No DB-driven backfill/reconciliation
24 orphan IDs	No DB existence validation
Restart did not recover c99	Restart re-seeded from incomplete snapshot
Some conversations can appear after opening/routing	Runtime events can introduce their summary
58 DBs remain healthy	Durable store is not the corrupted layer

8 — The c99 Forensic Anchor

The missing c99 conversation was the key case because it gave the investigation a concrete object that could be tested independently of the UI.

Field	Value
DB	c99ca0b3-a658-48ae-9dba-33eed0571a99.db
Title	is there any session manager in this codebase i made days ago can you find
Prompts / actions / steps	31 / 124 / 329
trajectory_id	3c037681-fcca-4eaa-a8ef-28f9f7e86aee
trajectory_type / source	4 / 1
Workspace	file:///c:/Users/Anurag/Desktop/KoreanDrama/Korean_insta
Comparison DB	686e30bc-6e67-4f84-b395-5b901354f926.db

Both c99 and the known-visible 686 conversation had structurally identical DB schemas, valid metadata, and successful SQLite checks. This weakened explanations based on a malformed conversation database.

These fields were cross-validated during reconstruction against the IDE-authored summary representation. The significance is deterministic derivability, not the timestamps themselves.

The sidecar incident

During early inspection, opening DBs without immutable=1 caused SQLite -wal and -shm sidecars to appear. They were zero-byte WAL files and roughly 32 KB SHM files across 55 DBs. Database mtimes remained unchanged and all 58 quick_check operations remained healthy. Immutable read access was then used. This was a tooling artifact, not the History root cause.

9 — Frontend Search and Visibility

The conversation picker reads trajectory summaries from its provider and combines them with external summaries. The search path can also call the native searchConversations bridge.

The native bridge exposes searchConversations, subscribeToSummaries, writeSummary and deleteSummary. The frontend therefore depends on the summary layer for conversation discovery.

B5u selector

B5u is a display selector. It sorts by lastUserInputTime, may exclude parent conversations under a feature flag, and requires a workspace URI intersection. It does not repair or delete persisted summary entries.

This produced an important caveat: the repaired set contains conversations across 13 workspaces. The UI can legitimately show only those belonging to the currently open workspace. That is a visibility rule, not evidence of data loss.

10 — Migration, CLI Roots and Startup Loading

10.1 Migration

Migration code was examined because it can bulk-load trajectories. The evidence showed a one-shot project migration guarded by state. Status 1 (STARTED) and 2 (COMPLETED) skip migration; 0 (UNSPECIFIED) and 3 (FAILED) permit a run. Existing migration state therefore does not establish a recurring History indexer.

10.2 CLI versus IDE

The existence of .gemini\antigravity-cli artifacts initially looked suspicious. The traced configuration instead showed distinct execution modes: IDE processes use --app_data_dir antigravity-ide with subclient_type ide, while standalone/offline CLI paths use separate state roots. CLI artifacts therefore could not be promoted to 'the IDE's source of truth' without evidence.

10.3 Startup hydration

NewDefaultManager → Manager.Init → RegisterTrajectories registers trajectory/store information. It does not populate the complete CascadeManager registry. Exhaustive map-assignment tracing showed updateStateForLoadedTrajectory as part of the actual load path. GetAllTrajectories simply reads the existing map.

This resolved an apparent contradiction: 'the DB exists' and 'the runtime does not know it' can both be true in a load-driven architecture.

11 — The Google Antigravity Expectation and What Actually Happened

At the beginning, there was a natural expectation that Antigravity's own agent environment would identify the product's internal failure and provide the root fix. That expectation was not fulfilled.

Early agent outputs made several confident claims that were later contradicted by direct source/binary evidence. The report records this carefully: it does not infer motive, intent, or why those outputs were produced. The engineering conclusion is only that the claims were not reliable root-cause evidence.

Examples of later-corrected claims included treating the 25 entries as an eviction boundary, attributing the problem to the CLI root, and assuming startup/migration behavior that was not present in the traced execution path.

Why the investigation moved to OpenCode / Big Pickle

Once the problem became a forensic source-tracing task, OpenCode was used as a more suitable exploration environment. It enabled broad local searches, symbol/call-site analysis, inspection of minified JavaScript, examination of native binary strings/functions, and iterative testing of hypotheses.

Kiro and Codex/Work were also used as supplementary investigation paths. Usage/token constraints interrupted those paths, so neither was treated as the decisive evidence source.

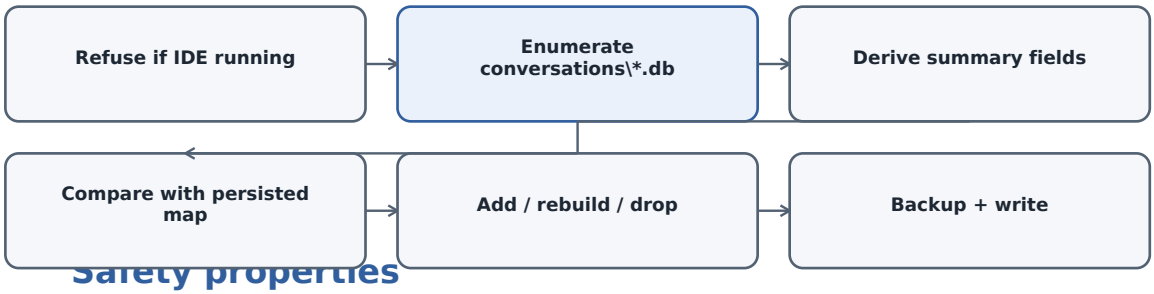
Tool contribution matrix

Tool/path	Contribution	Limit
PowerShell / filesystem	Files, counts, timestamps, DB integrity	Limited architectural visibility
Antigravity CLI	Initial bundle/path/symbol discovery	Early hypotheses required later verification
OpenCode / Big Pickle	Deep source/binary archaeology and call graphs	Still required local evidence
Kiro	Supplementary analysis	Token/usage interruption
Codex / Work	Additional engineering analysis	Usage-limit interruption
Recovery script	Deterministic repair and tests	Must adapt to future format changes

12 — Recovery Engineering

Recovery was designed around the authoritative conversation DB set rather than the broken summary map.

External reconciler



Safety properties

- Never modifies conversation DBs.
- Refuses to run while Antigravity is active.
- Backs up state before a write.
- Only writes when real drift exists.
- Preserves valid IDE-authored entries where appropriate.
- Removes orphan summary IDs.
- Rebuilds incomplete/corrupt summary entries from DB metadata.
- Is idempotent: a clean second run is a no-op.

Why not patch minified vendor code?

A permanent modification to minified workbench JavaScript or a compiled language-server binary would be tightly coupled to a particular product build. A product update could replace the patch or change internal offsets/serialization. The reconciler instead acts as an external integrity boundary and can be updated independently.

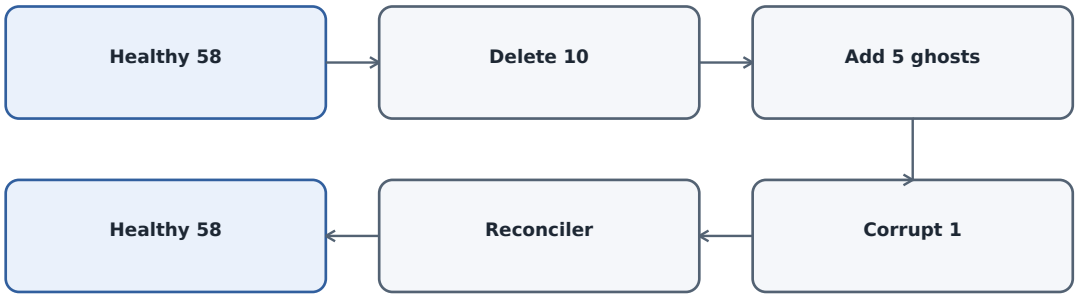
13 — Validation and Fault Injection

The recovery was not accepted after one successful reconstruction. It was tested against destructive drift and then against the real application lifecycle.

Validation	Result	What it proves
Wiped topic	58 reconstructed	Full rebuild from authoritative DBs
Cross-validation	All checks passed	Fields match expected IDE representation
10 deleted entries	10 restored	Missing-summary repair
5 ghost entries	5 removed	Orphan cleanup
1 corrupted entry	1 rebuilt	Repair of damaged representation
Valid entries retained	47	Non-damaged data preserved
Immediate second run	No-op	Idempotence
Real launch/shutdown/restart	58 before/during/after	Durable restart survival
User-created History Reconciliation Test	Visible	End-to-end UI verification

Fault-injection diagram

Controlled corruption → convergence

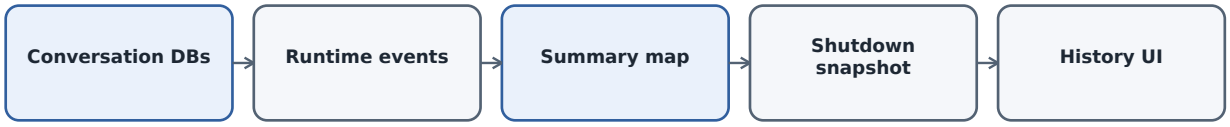


The restart test was decisive because the original failure involved shutdown persistence. The repaired state survived a real launch → clean shutdown → restart sequence with 58 entries before, during, and after. This demonstrated that the repair became the persisted seed rather than a temporary in-memory patch.

14 — Before / After Architecture

BEFORE: event-only consistency

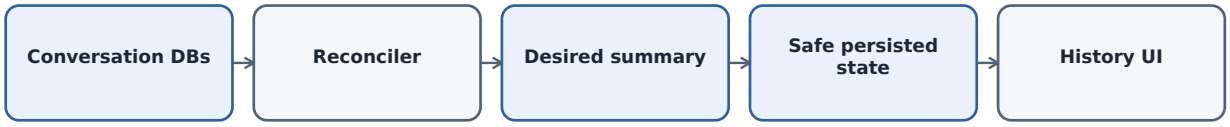
Before



The missing edge is the problem: there is no authoritative reconciliation from the DB set back into the summary map.

AFTER: explicit integrity control

After



The new architecture does not require patching Antigravity. It adds a consistency boundary that can detect and repair projection drift.

15 — Operational Lessons and Future Caveats

• The visible UI is a projection, not necessarily the source of truth.

• A small number of entries is not proof of an eviction policy.

• Runtime loading and durable indexing are separate concerns.

• Whole-map shutdown persistence can preserve an incomplete map.

• Recovery should start from the deepest authoritative source.

• Agent-generated explanations should remain hypotheses until backed by artifacts.

• External reconciliation is less coupled to minified vendor code than invasive patching.

• Workspace filtering must not be confused with durable index loss.

• Cross-profile assumptions can create a second integrity problem; the secondary profile was deliberately left unchanged.

• Future Antigravity storage-format changes may require schema updates; the reconciler should fail closed rather than silently write incompatible data.

Operational state after repair

The scheduled task 'Antigravity History Reconciler' runs at logon and hourly. The repair script is idempotent and only writes on actual drift. A pre-write rollback artifact was retained, and a conversation manifest/recovery report was generated.

Known caveats

• 58 conversations span 13 workspaces; current History visibility remains workspace-dependent.

• Conversation titles may initially reflect the first prompt because AI-generated titles are not stored in the DB.

• The secondary profile containing 19 phantom entries was not repaired because there was insufficient evidence to cross-populate it.

• Future releases may alter protobuf/schema fields or summary semantics.

16 — Technical Reference

Reference	Value
Affected conversation	c99ca0b3-a658-48ae-9dba-33eed0571a99
Known visible comparison	686e30bc-6e67-4f84-b395-5b901354f926
History topic	antigravityUnifiedStateSync.trajectorySummaries
Conversation store	%USERPROFILE%\.gemini\antigravity-ide\conversations*.db
Brain root	%USERPROFILE%\.gemini\antigravity-ide\brain\{cascadeId}
Global state	%APPDATA%\Antigravity IDE\User\globalStorage\state.vscdb
State table	ItemTable(key TEXT, value BLOB)
State key	chat.ChatSessionStore.index
Installed app	%LOCALAPPDATA%\Programs\Antigravity IDE\resources\app
Rollback	antigravity-recovery-prewrite-20261003-110718
Recovery outputs	RECOVERY_REPORT.md; AUTHORITATIVE_CONVERSATION_MANIFEST.csv
Scheduled task	Antigravity History Reconciler

Implementation fragments

Evidence handling rule: conversation DBs were preserved byte-for-byte against immutable backups; the reconciler does not modify them. Persistent changes were limited to the summary/index state and supporting recovery artifacts.

17 — Decision Log and Evidence Ledger

Decision	Reason
Use DBs as authoritative source	Complete readable trajectories; integrity checks passed.
Reject 25-entry eviction theory	Actual cleanup threshold is 100.
Separate CLI from IDE roots	Different execution modes and state paths.
Trace binaries and bundles	Visible behavior could not explain persistence divergence.
Use external reconciler	Avoid vendor patching and improve update resilience.
Leave secondary profile untouched	No evidence justified cross-profile repair.
Require idempotence	Integrity control must converge and become a no-op.

Evidence ledger

Claim	Classification
58 conversation DBs exist	PROVEN
58 DBs pass SQLite quick_check	PROVEN
c99 contains 329 steps	PROVEN
25 summary entries existed	PROVEN
24 summary IDs were orphans	PROVEN
57 DB IDs were absent from summary map	PROVEN
25 is the eviction cap	DISPROVEN
CLI root caused IDE loss	NOT SUPPORTED
Migration caused steady-state loss	DISPROVEN
CascadeManager bulk-loads all DBs at startup	DISPROVEN
B5u persistently deleted conversations	DISPROVEN
No steady-state DB reconciliation was found	PROVEN BY TRACED IMPLEMENTATION
58-entry reconstruction succeeds	PROVEN
Fault injection converges correctly	PROVEN
Restart preserves repaired state	PROVEN
User History test succeeds	PROVEN

18 — Final Assessment

The incident was not fundamentally a case of deleted conversation databases. It was a consistency failure between a healthy durable conversation store and a separately persisted History summary projection.

The complete investigation progressed from symptom → filesystem evidence → competing hypotheses → source/binary tracing → exact summary write path → deterministic reconstruction → fault injection → restart validation → user verification. Each stage narrowed uncertainty.

The initial expectation that Antigravity itself would supply the root cause and fix did not survive evidence. The successful approach was to treat all explanations—including early confident agent explanations—as provisional and require executable/source evidence before accepting them.

The repaired system now enforces the missing invariant: every authoritative conversation DB should have a valid History summary, and every persisted History summary should correspond to an authoritative conversation DB.

The reconciler is the local integrity control that enforces this invariant without modifying Antigravity's vendor bundles. It backs up before writes, refuses concurrent execution, repairs additions/removals/corruption, and converges to a stable state.

The recovery is validated at the application lifecycle level: the repaired projection survived clean shutdown and restart, and a newly created test conversation was visible in History. The remaining operational caveat is future format evolution; the reconciler must be revalidated after product updates.

Author's Closing Note

A personal note on the engineering lesson behind this investigation

Even though I use AI and AI tools for coding—and from the outside that can make the work look easy—I completely agree with the basic idea: if you give someone the tools, and they gain enough experience with those tools, they can use them much more effectively. But this investigation reminded me of a different question: what happens when the tool itself breaks?

That was the point where this stopped being ordinary AI-assisted coding. I had to go behind the editor itself. While reconstructing what was happening inside the VS Code-based architecture, I ended up learning how the editor, its state, its conversation storage, its runtime loading, its native components, and its persistence layers were actually connected. The tool stopped being something I could simply use and became the system I had to understand.

I also noticed something about the way I approach problems. I somehow keep ending up on the hardest route. Sometimes I deliberately choose a path that almost nobody would want to take; sometimes I arrive there without even intending to. I try to find the easy way, but somehow I end up tracing the thing all the way down. In this case, that meant going far beyond “the history is missing” and asking where the data actually lived, which layer was authoritative, what was loaded into memory, what was persisted, and what code was responsible for writing the broken state.

So I do not consider this simply a case of using an AI tool to write code. I used AI heavily, but I also read, searched, traced, debugged, questioned assumptions, inspected binaries and application bundles, compared databases, reconstructed state, and tested the result. The hardest part was not writing the fix. The hardest part was finding the root cause.

That distinction became very clear during this session. Every time I thought I had found the root cause, GPT pushed back and questioned whether the evidence actually proved it. Sometimes the explanation looked convincing, but another piece of evidence did not fit. That forced the investigation deeper. The process was not “find something that sounds right and patch it”; it was repeatedly asking whether the proposed cause could explain all of the observed evidence.

The time spent makes that difference especially obvious. The investigation lasted roughly nine hours, with about 8½ hours spent finding and tracing the root cause and roughly 30 minutes spent implementing the actual fix. The larger session was close to twelve hours overall once the surrounding work, validation, and recovery were included. That time gap is probably the clearest lesson in this entire report: in difficult engineering problems, the implementation of the fix can be short compared with the work required to understand why the system is broken.

I also restarted Antigravity after the repair and verified that the repaired sessions remained available. That mattered to me because a fix that works only until the next restart is not really a fix. The persistence behavior had to be tested, not assumed.

At several points I thought about stopping. The original task sounded simple: recover important company AI-model-related data from an IDE that was no longer showing it correctly. If the data had not been important, I probably would have been satisfied with a patch and moved on. But because the underlying conversations mattered, I kept going until I understood the storage and

persistence path well enough to build a safer solution.

This session also became one of my longest focused debugging sessions. I have previously had a session of around 36 hours straight, with breaks, but this one still reached almost twelve hours and required sustained investigation rather than just continuous coding. By the end, what started as a missing-history problem had turned into a practical lesson in reverse engineering, debugging methodology, persistence design, and evidence-based reasoning.

> The part I will remember is not the final script. It is the difference between knowing how to use a tool and knowing how to investigate the tool when the tool itself becomes the problem.

— End of author's note —